



UNIVERSIDAD PRIVADA DE TACNA

Facultad de Ingeniería
Escuela Profesional de Ingeniería de Sistemas

PROYECTO DE ANTIVIRUS · CALIDAD Y PRUEBAS DE SOFTWARE

Estándar de Programación

Sistema RustGuard Antivirus

Versión 2.0 · Julio 2026

Docente:	Mag. Patrick Cuadros Quiroga
Curso:	Calidad y Pruebas de Software
Ciclo:	Séptimo — Ingeniería de Sistemas

EQUIPO RUSTGUARD

LLica Mamani, Jimmy Mijair (2023076789)
Sierra Ruiz, Iker Alberto (2023077090)

CONTROL DE VERSIONES DEL DOCUMENTO

El presente registro documenta la trazabilidad de cambios efectuados sobre *Estándar de Programación*, en conformidad con los protocolos de aseguramiento de calidad del proyecto RustGuard.

CONTROL DE VERSIONES				
Versión	Hecha por	Revisada por	Aprobada por	Fecha
1.0	Equipo RustGuard	Mag. Patrick Cuadros Quiroga	Equipo RustGuard	04/07/2026
2.0	Equipo RustGuard	Mag. Patrick Cuadros Quiroga	Equipo RustGuard	04/07/2026

ÍNDICE GENERAL

1. [Introducción](#)
2. [Estándares Generales de Nomenclatura](#)
3. [Estándares para el Frontend \(React 19, TailwindCSS, Vite\)](#)
4. [Estándares para el Backend de Escritorio \(Electron, Node.js\)](#)
5. [Estándares de Pruebas \(Vitest, Playwright, Gherkin\)](#)
6. [Estándares de Infraestructura \(Terraform, GitHub Actions\)](#)
7. [Manejo de Errores y Seguridad IPC](#)

1. Introducción

El sistema **RustGuard Antivirus** está construido bajo una arquitectura de proceso dual utilizando **Electron** (Backend) y **React** (Frontend). Mantener la calidad del software (SQA) requiere la definición de un **Estándar de Programación** estricto que alinee las convenciones semánticas, estructurales y las prácticas de testing (BDD, E2E) aplicadas por los ingenieros de calidad y desarrollo.

2. Estándares Generales de Nomenclatura

Independientemente del módulo del proyecto, RustGuard adopta las siguientes reglas transversales:

- **Idioma de Código:** El código fuente (nombres de variables, funciones, clases, archivos `.feature` de BDD, y flujos de Terraform) se escribirá íntegramente en **Inglés** (Ej. `scanDirectory()`, `ScanResult`, `main.tf`).
 - **Idioma de la UI y Logs Finales:** Todos los textos visibles por el usuario (Alertas, componentes React, Output del sistema) deben redactarse en **Español** (UTF-8).
 - **Variables Estáticas (Constantes):** Deben declararse en `UPPER_SNAKE_CASE` (Ej. `MAX_FILE_SIZE`).
 - **Reglas de Linting:** Todo el proyecto obedece las reglas estrictas de `eslint.config.js`. Cualquier advertencia (*warning*) debe resolverse antes de un *Commit*.
-

3. Estándares para el Frontend (React 19, TailwindCSS, Vite)

El código alojado en el directorio `/src/` sigue patrones modernos de desarrollo web reactivo.

- **Componentes Funcionales:** Uso mandatorio de *React Hooks*. Está prohibido el uso de componentes basados en clases.
 - **Nomenclatura de Archivos:** `PascalCase` para componentes (Ej. `DashboardPanel.jsx`) y `camelCase` para utilidades (Ej. `formatDate.js`).
 - **Estilizado (TailwindCSS 4):**
 - Las clases utilitarias deben aplicarse directamente en la propiedad `className`.
 - Se prohíbe el uso de estilos en línea (`style={{ color: 'red' }}`) salvo para valores dinámicos calculados en tiempo de ejecución.
 - **Separación de Responsabilidades:** La lógica compleja debe abstraerse en *Custom Hooks* (Ej. `useScanner.js`) para mantener los componentes de UI limpios y enfocados en el renderizado.
-

4. Estándares para el Backend de Escritorio (Electron, Node.js)

El código alojado en el directorio `/electron/` opera con privilegios de sistema operativo y controla al motor ClamAV.

- **Nomenclatura:** Archivos y funciones deben escribirse en `camelCase`.
 - **Manejo de Asincronía:** Es obligatorio el uso de `async / await` en lugar de *Callbacks* tradicionales para evitar el *Callback Hell* al operar con el sistema de archivos (`fs.promises`).
 - **Uso de Child Process:** Las llamadas al ejecutable `clamscan` deben realizarse exclusivamente mediante `child_process.spawn()` (nunca `exec()` para evitar inyecciones de shell con nombres de archivo malformados).
-

- **Canal IPC (Inter-Process Communication):** Toda petición de React hacia Node debe fluir a través del `contextBridge` definido en `preload.js`. Se prohíbe exponer directamente objetos de Node (`require`, `process`) al hilo de renderizado.
-

5. Estándares de Pruebas (Vitest, Playwright, Gherkin)

La calidad está garantizada mediante un enfoque híbrido BDD/E2E y unitario.

- **Pruebas Unitarias (Vitest):**
 - Ubicadas en `/src/test/`.
 - Deben cubrir al menos el **40%** del código lógico para superar la validación de CI.
 - *Nomenclatura:* `[nombre].test.js`.
 - **Pruebas de Mutación (Stryker):**
 - Las pruebas unitarias deben diseñarse para detectar fallos si el código es alterado por Stryker. Asegurar aserciones (`expect`) robustas.
 - **Pruebas BDD (Playwright + Gherkin):**
 - Los escenarios deben definirse en archivos `.feature` dentro de `/src/features/` usando lenguaje Gherkin estricto (`Given`, `When`, `Then`).
 - Las definiciones de pasos (*Step Definitions*) deben usar selectores accesibles (ej. `getByRole`, `getByText`) preferentemente sobre selectores CSS rígidos.
-

6. Estándares de Infraestructura (Terraform, GitHub Actions)

- **Pipelines CI/CD (`.github/workflows/`):**
 - Nombrar los *jobs* de forma descriptiva (Ej. `run-static-analysis`, `build-electron-app`).
 - Utilizar matrices para pruebas paralelas cuando sea necesario.
 - **Terraform (`/terraform/`):**
 - Formateo obligatorio antes de confirmar cambios: `terraform fmt`.
 - Separar definición de recursos (`main.tf`), variables (`variables.tf`) y salidas (`outputs.tf`).
 - Seguir convenciones `snake_case` para el nombramiento de recursos cloud.
-

7. Manejo de Errores y Seguridad IPC

- **Fallas Silenciosas (Silent Fails):** Nunca ocultar un error. Si una operación falla, usar `console.error` o retornar un objeto serializable con un código de error a la UI. No usar `catch (e) {}` vacío.
 - **Validación de Entradas:** Antes de ejecutar `clamscan`, el backend de Electron debe verificar explícitamente mediante expresiones regulares que la ruta del archivo inyectada a través de IPC no contiene caracteres de escape maliciosos.
 - **Dependencias Seguras:** Ejecutar siempre correcciones de seguridad sugeridas por el flujo *Snyk* de la pipeline para mantener `package-lock.json` blindado contra librerías vulnerables (CVEs).
-